FHIR Server Access Instructions

Server Information

- **FHIR Endpoint:** `https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4`
- **Username:** `fhiruser`
- **Password:** `BmI512@ccess`
- **Authentication:** HTTP Basic Authentication
- **Server Type:** IBM FHIR Server (Linux for Health)
- **FHIR Version:** R4

Table of Contents

Using cURL

Basic Authentication Setup

The server uses HTTP Basic Authentication. You can provide credentials in two ways:

Method 1: Using -u flag (Recommended)

```
curl -u fhiruser:BmI512@ccess \
  -H "Accept: application/fhir+json" \
  -k \
  https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4/metadata
```

Method 2: Using Authorization Header

First, encode credentials in Base64:

```
echo -n "fhiruser:BmI512@ccess" | base64
# Output: ZmhpcnVzZXI6Qm1JNTEyQGNjZXNz
```

Then use in request:

```
curl -H "Authorization: Basic ZmhpcnVzZXI6Qm1JNTEyQGNjZXNz" \
  -H "Accept: application/fhir+json" \
  -k \
  https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4/metadata
```

Common cURL Operations

1. Get Server Metadata (CapabilityStatement)

```
curl -u fhiruser:BmI512@ccess \
  -H "Accept: application/fhir+json" \
  -k \
  https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4/metadata
```

2. Search for Patients

```
curl -u fhiruser:BmI512@ccess \
  -H "Accept: application/fhir+json" \
  -k \
  https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4/Patient
```

3. Get Specific Patient by ID

```
curl -u fhiruser:BmI512@ccess \
  -H "Accept: application/fhir+json" \
  -k \
  https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4/Patient/
example-id
```

4. Create a New Patient (POST)

```
curl -u fhiruser:BmI512@ccess \
  -X POST \
  -H "Content-Type: application/fhir+json" \
  -H "Accept: application/fhir+json" \
  -k \
  -d '{
    "resourceType": "Patient",
    "name": [
      {
        "family": "Doe",
        "given": ["John"]
      }
    ],
    "gender": "male",
    "birthDate": "1990-01-01"
  }' \
  https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4/Patient
```

5. Submit Transaction Bundle

```
curl -u fhiruser:BmI512@ccess \
  -X POST \
  -H "Content-Type: application/fhir+json" \
  -H "Accept: application/fhir+json" \
  -k \
  -d @synthetic_fhir_r4_transaction_bundle_20.json \
  https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4/
```

Note: Transaction bundles must be posted to the root endpoint (`/`), not `/Bundle` .

6. Update a Patient (PUT)

```
curl -u fhiruser:BmI512@ccess \
  -X PUT \
  -H "Content-Type: application/fhir+json" \
  -H "Accept: application/fhir+json" \
  -k \
  -d '{
    "resourceType": "Patient",
    "id": "example-id",
    "name": [
      {
        "family": "Doe",
        "given": ["Jane"]
      }
    ],
    "gender": "female",
    "birthDate": "1990-01-01"
  }' \
  https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4/Patient/example-id
```

7. Delete a Patient

```
curl -u fhiruser:BmI512@ccess \
  -X DELETE \
  -k \
  https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4/Patient/example-id
```

8. Search with Parameters

```
# Search patients by family name
curl -u fhiruser:BmI512@ccess \
```

```
  -H "Accept: application/fhir+json" \
  -k \
  "https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4/
Patient?family=Doe"

# Search patients by birthdate
curl -u fhiruser:BmI512@ccess \
  -H "Accept: application/fhir+json" \
  -k \
  "https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4/
Patient?birthdate=1990-01-01"
```

cURL Flags Explained

- `-u username:password` - Provides Basic Auth credentials
- `-H "Header: Value"` - Adds HTTP header to request
- `-k` or `--insecure` - Disables SSL certificate verification (required for self-signed certs)
- `-X METHOD` - Specifies HTTP method (GET, POST, PUT, DELETE)
- `-d` or `--data` - Sends data in request body
- `-d @filename` - Sends file contents as request body
- `--verbose` or `-v` - Shows detailed request/response information

Using Python with requests

Installation

```
pip install requests python-dotenv
```

Basic Setup

Method 1: Using Environment Variables (Recommended)

Create or use existing `.env` file:

```
FHIR_SERVER_URL=https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/
api/v4
FHIR_USERNAME=fhiruser
FHIR_PASSWORD=BmI512@ccess
VERIFY_SSL=false
```

Python code:

```
import os
import requests
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

# Configuration
FHIR_BASE_URL = os.getenv('FHIR_SERVER_URL')
USERNAME = os.getenv('FHIR_USERNAME')
PASSWORD = os.getenv('FHIR_PASSWORD')
VERIFY_SSL = os.getenv('VERIFY_SSL', 'true').lower() == 'true'

# Create session with authentication
session = requests.Session()
session.auth = (USERNAME, PASSWORD)
session.verify = VERIFY_SSL
session.headers.update({
    'Accept': 'application/fhir+json',
    'Content-Type': 'application/fhir+json'
})
```

Method 2: Hardcoded Credentials (Not Recommended for Production)

```
import requests

# Configuration
FHIR_BASE_URL = 'https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/
api/v4'
```

```
USERNAME = 'fhiruser'
PASSWORD = 'BmI512@ccess'

# Create session
session = requests.Session()
session.auth = (USERNAME, PASSWORD)
session.verify = False  # Disable SSL verification for self-signed certs
session.headers.update({
    'Accept': 'application/fhir+json',
    'Content-Type': 'application/fhir+json'
})

# Suppress SSL warnings
import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
```

Common Python Operations

1. Get Server Metadata

```
import requests
from requests.auth import HTTPBasicAuth

# Configuration
base_url = 'https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/
v4'
auth = HTTPBasicAuth('fhiruser', 'BmI512@ccess')

# Get metadata
response = requests.get(
    f'{base_url}/metadata',
    auth=auth,
    headers={'Accept': 'application/fhir+json'},
    verify=False
)

print(f"Status: {response.status_code}")
print(f"Response: {response.json()}")
```

2. Search for Patients

```
# Search all patients
response = session.get(f'{FHIR_BASE_URL}/Patient')

if response.status_code == 200:
    bundle = response.json()
    print(f"Found {bundle.get('total', 0)} patients")
    for entry in bundle.get('entry', []):
        patient = entry['resource']
        print(f"Patient ID: {patient['id']}")
else:
    print(f"Error: {response.status_code} - {response.text}")
```

3. Get Patient by ID

```
patient_id = 'example-id'
response = session.get(f'{FHIR_BASE_URL}/Patient/{patient_id}')

if response.status_code == 200:
    patient = response.json()
    print(f"Patient: {patient}")
else:
    print(f"Error: {response.status_code}")
```

4. Create a New Patient

```
import json

# Patient resource
new_patient = {
    "resourceType": "Patient",
    "name": [
        {
            "family": "Doe",
            "given": ["John"]
```

```
        }
    ],
    "gender": "male",
    "birthDate": "1990-01-01",
    "active": True
}

# POST request
response = session.post(
    f'{FHIR_BASE_URL}/Patient',
    json=new_patient
)

if response.status_code in [200, 201]:
    created_patient = response.json()
    print(f"Created patient with ID: {created_patient['id']}")
    print(f"Location: {response.headers.get('Location')}")
else:
    print(f"Error: {response.status_code} - {response.text}")
```

5. Submit Transaction Bundle

```
import json

# Load bundle from file
with open('synthetic_fhir_r4_transaction_bundle_20.json', 'r') as f:
    bundle = json.load(f)

# POST to root endpoint (not /Bundle!)
response = session.post(
    FHIR_BASE_URL + '/',  # Note: posting to root endpoint
    json=bundle
)

if response.status_code == 200:
    result_bundle = response.json()

    # Count successes and failures
    successes = 0
    failures = 0
```

```
    for entry in result_bundle.get('entry', []):
        status = entry.get('response', {}).get('status', '')
        if status.startswith('2'):  # 2xx status codes
            successes += 1
        else:
            failures += 1

    print(f"Transaction completed:")
    print(f"  Successful: {successes}")
    print(f"  Failed: {failures}")
else:
    print(f"Error: {response.status_code} - {response.text}")
```

6. Update a Patient

```
# Get existing patient
patient_id = 'example-id'
response = session.get(f'{FHIR_BASE_URL}/Patient/{patient_id}')

if response.status_code == 200:
    patient = response.json()

    # Modify patient
    patient['name'][0]['given'] = ['Jane']
    patient['gender'] = 'female'

    # PUT request
    update_response = session.put(
        f'{FHIR_BASE_URL}/Patient/{patient_id}',
        json=patient
    )

    if update_response.status_code == 200:
        print("Patient updated successfully")
    else:
        print(f"Update failed: {update_response.status_code}")
```

7. Delete a Patient

```python
patient_id = 'example-id'
response = session.delete(f'{FHIR_BASE_URL}/Patient/{patient_id}')

if response.status_code in [200, 204]:
    print("Patient deleted successfully")
else:
    print(f"Error: {response.status_code}")
```

8. Search with Parameters

```python
# Search by family name
response = session.get(
    f'{FHIR_BASE_URL}/Patient',
    params={'family': 'Doe'}
)

# Search by birthdate
response = session.get(
    f'{FHIR_BASE_URL}/Patient',
    params={'birthdate': '1990-01-01'}
)

# Multiple search parameters
response = session.get(
    f'{FHIR_BASE_URL}/Patient',
    params={
        'family': 'Doe',
        'gender': 'male',
        'birthdate': 'gt1980-01-01'  # greater than 1980-01-01
    }
)

if response.status_code == 200:
    bundle = response.json()
    print(f"Found {len(bundle.get('entry', []))} patients")
```

9. Error Handling with OperationOutcome

```python
def handle_fhir_response(response):
    """Handle FHIR response and parse OperationOutcome for errors"""

    if response.status_code in [200, 201]:
        return response.json()

    # Parse error response
    try:
        outcome = response.json()
        if outcome.get('resourceType') == 'OperationOutcome':
            issues = outcome.get('issue', [])
            for issue in issues:
                severity = issue.get('severity', 'unknown')
                code = issue.get('code', 'unknown')
                details = issue.get('diagnostics', 'No details')
                print(f"[{severity.upper()}] {code}: {details}")
    except:
        print(f"Error: {response.status_code} - {response.text}")

    return None

# Usage
response = session.post(f'{FHIR_BASE_URL}/Patient', json=patient_data)
result = handle_fhir_response(response)
if result:
    print("Operation successful")
```

10. Complete Example with Retry Logic

```python
import requests
import time
from requests.adapters import HTTPAdapter
from requests.packages.urllib3.util.retry import Retry

def create_fhir_session(base_url, username, password, verify_ssl=False):
    """Create a requests session with retry logic"""

    session = requests.Session()
    session.auth = (username, password)
    session.verify = verify_ssl
```

```
    session.headers.update({
        'Accept': 'application/fhir+json',
        'Content-Type': 'application/fhir+json'
    })

    # Configure retry strategy
    retry_strategy = Retry(
        total=3,
        backoff_factor=1,
        status_forcelist=[429, 500, 502, 503, 504],
        allowed_methods=["HEAD", "GET", "OPTIONS", "POST", "PUT"]
    )

    adapter = HTTPAdapter(max_retries=retry_strategy)
    session.mount("http://", adapter)
    session.mount("https://", adapter)

    return session

# Usage
session = create_fhir_session(
    base_url='https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/
api/v4',
    username='fhiruser',
    password='BmI512@ccess',
    verify_ssl=False
)

# Suppress SSL warnings
import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
```

Common Operations

Resource Types Supported

The server supports standard FHIR R4 resources including:

- `Patient`

- `Observation`
- `Encounter`
- `Condition`
- `MedicationRequest`
- `Practitioner`
- `Organization`
- And all other FHIR R4 resources

Search Parameters

FHIR supports rich search capabilities:

```
# Date ranges
?birthdate=gt1980-01-01                    # Greater than
?birthdate=lt2000-01-01                    # Less than
?birthdate=ge1980-01-01                    # Greater or equal
?birthdate=le2000-01-01                    # Less or equal

# Text search
?family=Doe                                 # Exact match
?family:contains=Do                        # Partial match

# Multiple values (OR)
?gender=male,female

# Multiple parameters (AND)
?family=Doe&gender=male

# Chaining (search by related resource)
?patient.name=Doe

# Reverse chaining
?_has:Observation:patient:code=12345-6

# Include related resources
?_include=Patient:organization
?_revinclude=Observation:patient

# Pagination
?_count=10&_page=2
```

Headers Reference

Required headers for FHIR operations:

```
Accept: application/fhir+json
Content-Type: application/fhir+json
Authorization: Basic <base64-credentials>
```

Troubleshooting

SSL Certificate Errors

Problem: SSL certificate verification fails

Solution:

- **cURL:** Use `-k` or `--insecure` flag
- **Python:** Set `verify=False` and suppress warnings:

```
import urllib3
urllib3.disable_warnings(urllib3.exceptions.InsecureRequestWarning)
```

401 Unauthorized

Problem: Authentication failed

Causes:

- Invalid credentials
- Missing Authorization header
- Incorrect Base64 encoding

Solution:

- Verify credentials: `fhiruser` / `BmI512@ccess`
- Check Authorization header format: `Basic <base64>`
- Test encoding: `echo -n "fhiruser:BmI512@ccess" | base64`

403 Forbidden

Problem: User lacks permissions for operation

Solution:
- Verify user has appropriate role/permissions on server
- Check server configuration for allowed interactions

404 Not Found

Problem: Resource doesn't exist

Solution:
- Verify resource ID is correct
- Check resource type in URL matches actual resource

422 Unprocessable Entity

Problem: Resource validation failed

Solution:
- Check response `OperationOutcome` for specific errors
- Ensure required fields are present
- Validate resource against FHIR R4 specification

Transaction Bundle Endpoint

CRITICAL: Transaction bundles must be posted to root endpoint:

```
# CORRECT
curl -X POST ... https://server/fhir-server/api/v4/
```

```
# WRONG
curl -X POST ... https://server/fhir-server/api/v4/Bundle
```

Connection Timeout

Problem: Request times out

Solution:

- Increase timeout: `curl --max-time 120` or `requests.get(..., timeout=120)`
- Check server status: `docker ps` and `docker logs`
- Consider splitting large bundles

Debugging Tips

1. **Verbose cURL output:**

```
curl -v -u fhiruser:BmI512@ccess ...
```

2. **Python response details:**

```
print(f"Status: {response.status_code}")
print(f"Headers: {response.headers}")
print(f"Body: {response.text}")
```

3. **Check server logs:**

```
ssh jbalsamo@lfh-fhir.eastus2.cloudapp.azure.com
docker logs fhir-server --tail 100
```

Additional Resources

- **FHIR R4 Specification:** https://hl7.org/fhir/R4/

- **IBM FHIR Server Documentation:** https://ibm.github.io/FHIR/
- **FHIR RESTful API:** https://hl7.org/fhir/R4/http.html
- **Search Parameters:** https://hl7.org/fhir/R4/search.html

Quick Reference

```
# Base URL
https://lfh-fhir.eastus2.cloudapp.azure.com:9443/fhir-server/api/v4

# Credentials
Username: fhiruser
Password: BmI512@ccess

# Basic cURL Template
curl -u fhiruser:BmI512@ccess \
  -H "Accept: application/fhir+json" \
  -k \
  [URL]

# Basic Python Template
import requests
session = requests.Session()
session.auth = ('fhiruser', 'BmI512@ccess')
session.verify = False
session.headers.update({'Accept': 'application/fhir+json'})
response = session.get('[URL]')
```